O&G · 7 — Multiphase flow: Buckley–Leverett waterflood displacement

Every earlier O&G notebook treated a single flowing phase. Waterflooding — injecting water to push oil toward a producer — is inherently two-phase, and its classic theory is the **Buckley–Leverett** equation. For 1D, incompressible, immiscible displacement with no capillary pressure, the water saturation S_w obeys the hyperbolic conservation law

$$\frac{\partial S_w}{\partial t} + v\, f_w'(S_w)\, \frac{\partial S_w}{\partial x} = 0$$

where $f_w(S_w)$ is the **fractional flow** of water (the fraction of total flow that is water at a given saturation) and v is the interstitial velocity. Because f_w is S-shaped, naively following each saturation's characteristic produces a triple-valued, unphysical profile — resolved by **Welge's tangent construction**, which replaces the unphysical part with a sharp saturation shock. We build f_w from Corey relative permeabilities, find the shock with `Roots.jl`, and get its slope with `ForwardDiff.jl` — the same automatic-differentiation tool used for curve fitting elsewhere in this repository, now differentiating a closed-form flow function instead of an ODE solve.

Everything below is dimensionless (saturations, fractional flow, and pore volumes injected, Q_i) — the shock construction and breakthrough timing are unit-free by nature, so there's no field-unit conversion to get wrong.

```julia
using ForwardDiff, Roots, PlutoUI, Plots
```

1. Relative permeability and fractional flow

Corey-type curves: water and oil relative permeability ramp from zero at their respective irreducible saturations.

```
krw (generic function with 1 method)
```

```julia
function krw(Sw, Swc, Sor, krw_max, nw)
    Sw <= Swc && return 0.0
    Sw >= 1 - Sor && return krw_max
    Swn = (Sw - Swc) / (1 - Swc - Sor)
    return krw_max * Swn^nw
end
```

kro (generic function with 1 method)

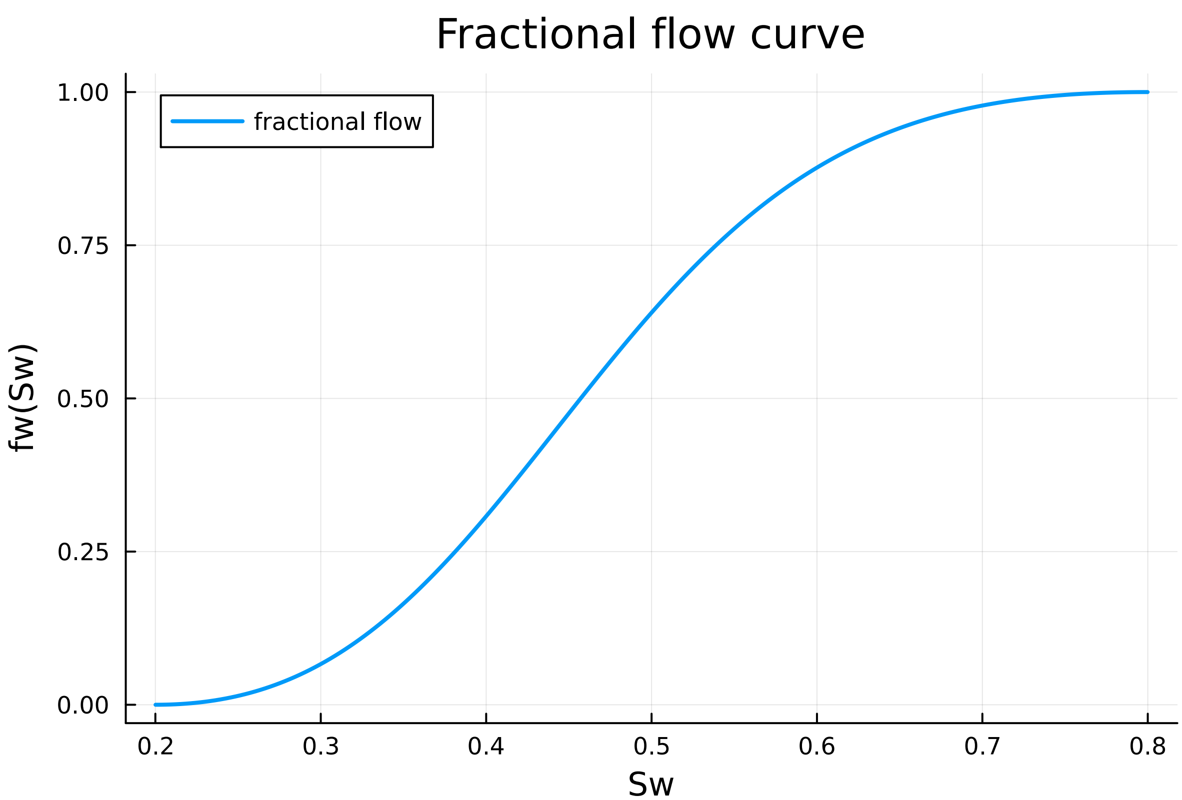
```
function kro(Sw, Swc, Sor, kro_max, no)
    Sw <= Swc && return kro_max
    Sw >= 1 - Sor && return 0.0
    Son = (1 - Sw - Sor) / (1 - Swc - Sor)
    return kro_max * Son^no
end
```

fw (generic function with 1 method)

```
function fw(Sw, p)
    Swc, Sor, krw_max, kro_max, nw, no, μw, μo = p
    kw = krw(Sw, Swc, Sor, krw_max, nw)
    ko = kro(Sw, Swc, Sor, kro_max, no)
    return (kw / μw) / (kw / μw + ko / μo)
end
```

▸ (0.2, 0.2)

```
begin
    # Swc, Sor, krw_max, kro_max, nw, no, μw (cp), μo (cp)
    p_bl = (0.2, 0.2, 0.4, 0.9, 2.0, 2.0, 0.5, 2.0)
    Swc, Sor = p_bl[1], p_bl[2]
end
```




```
begin
    Sw_grid = range(Swc + 1e-4, 1 - Sor - 1e-4; length=200)
    plot(Sw_grid, fw.(Sw_grid, Ref(p_bl)); xlabel="Sw", ylabel="fw(Sw)", label="fractional flow",
        title="Fractional flow curve", lw=2)
end
```

2. Welge's tangent construction: finding the shock front

The physically valid shock saturation S_{wf} is where the straight line from $(S_{wc}, 0)$ is *tangent* to the f_w curve — i.e. where the chord slope equals the local slope:

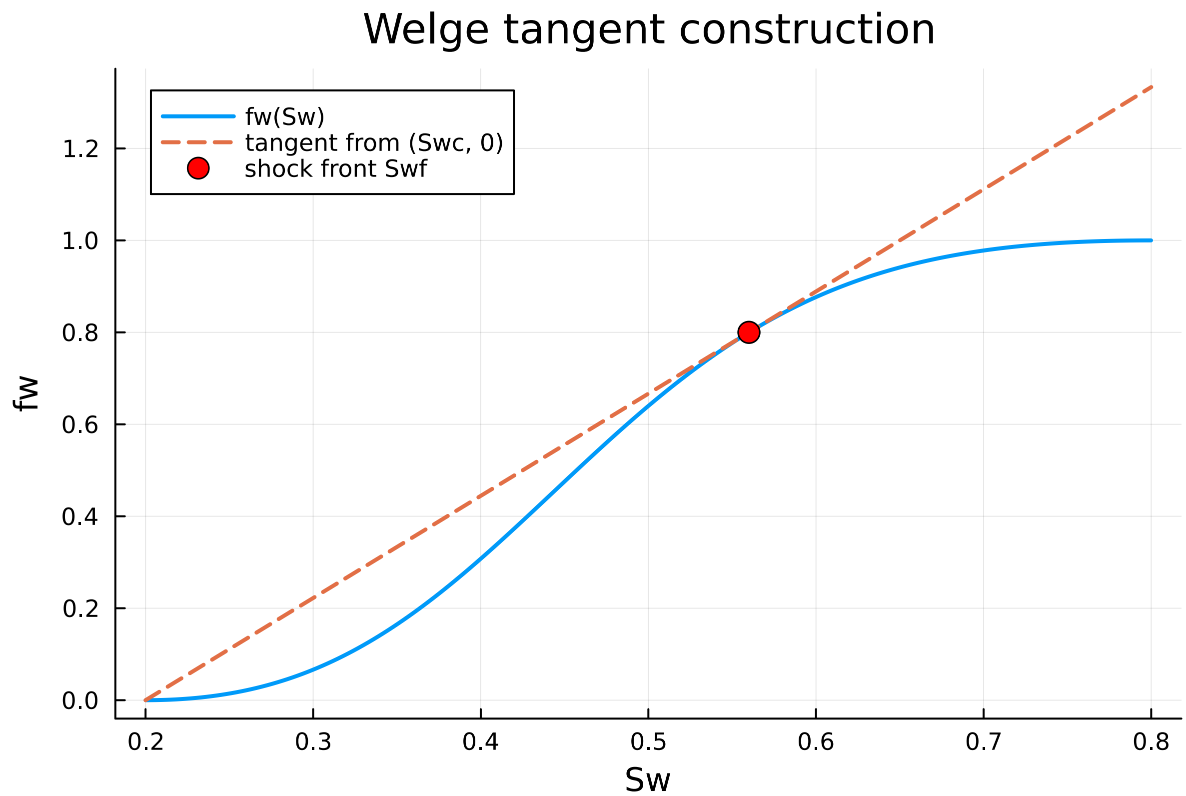
$$\frac{f_w(S_{wf})}{S_{wf} - S_{wc}} = f'_w(S_{wf})$$

```
dfw (generic function with 1 method)
```

```julia
dfw(Sw, p) = ForwardDiff.derivative(s -> fw(s, p), Sw)
```

```
(Sw_front = 0.56, fw_front = 0.8, dfw_front = 2.22222)
```

```julia
begin
    tangent_gap(Sw) = fw(Sw, p_bl) / (Sw - Swc) - dfw(Sw, p_bl)
    Sw_front = find_zero(tangent_gap, (Swc + 1e-3, 1 - Sor - 1e-3))
    fw_front = fw(Sw_front, p_bl)
    dfw_front = dfw(Sw_front, p_bl)
    (Sw_front=Sw_front, fw_front=fw_front, dfw_front=dfw_front)
end
```




```
begin
    plot(Sw_grid, fw.(Sw_grid, Ref(p_bl)); label="fw(Sw)", xlabel="Sw", ylabel="fw", lw=2,
        title="Welge tangent construction")
    plot!([Swc, 1 - Sor], [0.0, dfw_front * (1 - Sor - Swc)]; label="tangent from (Swc, 0)", lw=2, ls=:dash)
    scatter!([Sw_front], [fw_front]; label="shock front Swf", ms=6, color=:red)
end
```

3. Breakthrough and the average saturation behind the front

The front's characteristic velocity is proportional to $f_w'(S_{wf})$, so it reaches the outlet (dimensionless position $x_D = 1$) after $Q_{i,bt} = 1/f_w'(S_{wf})$ pore volumes injected. Welge's method also gives the average water saturation left behind the front with no extra integration.

```
(Qi_breakthrough = 0.45, Sw_avg_behind_front = 0.65)
```

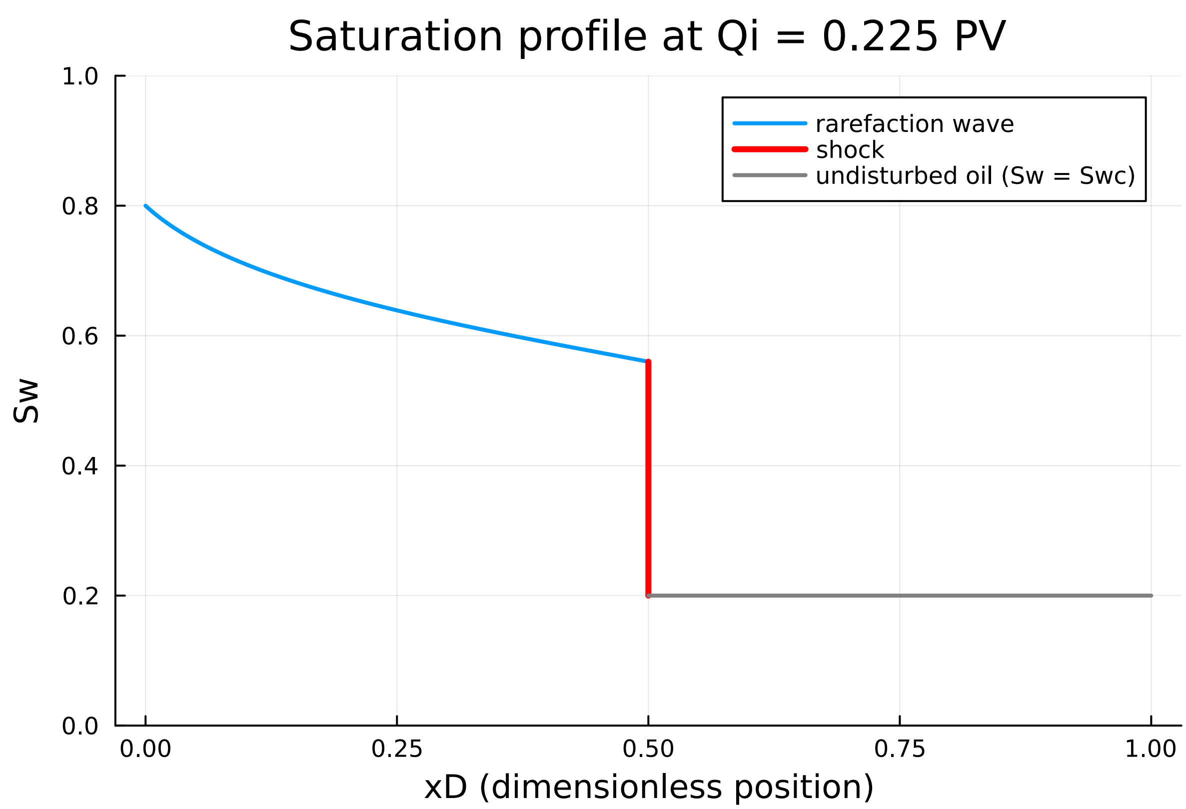
```
begin
    Qi_breakthrough = 1 / dfw_front
    Sw_avg_behind = Sw_front + (1 - fw_front) / dfw_front
    (Qi_breakthrough=Qi_breakthrough, Sw_avg_behind_front=Sw_avg_behind)
end
```

4. The saturation profile ahead of breakthrough

Behind the shock (for saturations from S_{wf} up to the injection-face value), each saturation travels at its own characteristic speed $x_D(S_w) = Q_i\, f_w'(S_w)$ — a valid, single-valued *rarefaction* wave. Ahead of the shock, only the original connate water S_{wc} is present. Drag Q_i to watch the flood front advance toward the outlet at $x_D = 1$.

0.5

```
@bind Qi_frac Slider(0.05:0.05:0.95, default=0.5, show_value=true)
```




```
begin
    Qi = Qi_frac * Qi_breakthrough   # stays pre-breakthrough: front hasn't reached the outlet yet

    Sw_behind = range(Sw_front, 1 - Sor; length=100)
    xD_behind = Qi .* dfw.(Sw_behind, Ref(p_bl))
    xD_front = Qi * dfw_front

    plot(xD_behind, Sw_behind; label="rarefaction wave", lw=2, xlabel="xD (dimensionless position)",
        ylabel="Sw", title="Saturation profile at Qi = $(round(Qi, digits=3)) PV", ylims=(0, 1))
    plot!([xD_front, xD_front], [Swc, Sw_front]; label="shock", lw=3, color=:red)
    plot!([xD_front, 1.0], [Swc, Swc]; label="undisturbed oil (Sw = Swc)", lw=2, color=:gray)
end
```

Takeaways

- The Buckley–Leverett profile isn't solved with an ODE or PDE integrator at all here — `Roots.jl` finds the shock, and `ForwardDiff.jl` supplies the exact slope of a closed-form fractional-flow curve, the same automatic-differentiation building block used for curve fitting elsewhere in this repository.
- Working entirely in dimensionless saturations and pore-volumes-injected sidesteps unit-conversion risk altogether — the shock location and breakthrough timing are unit-free by construction.
- This is the textbook (Welge) construction for the simplest case — incompressible, 1D, no capillary pressure or gravity. A full reservoir simulator adds those effects numerically (finite-volume/finite-difference), but the fractional-flow curve built here is exactly the physics such a simulator has to get right first.